

Afondamento nas competencias profesionais

**DOCUMENTACIÓN DE ARQUITECTURA
DE SOFTWARE Y REQUISITOS
FUNCIONALES**

CAFETERÍA MEDIANTE HILOS DE JAVA

Jacobo Pérez de Torres

1 Introducción y descripción del problema

Esta aplicación simula el funcionamiento de una cafetería empleando una interfaz generada en JavaFX y el módulo de Thread de Java. Existen dos camareros (Hilos) que atienden concurrentemente a clientes (También hilos). Los clientes llegan a la cafetería siguiendo el orden de la lista y con un retraso aleatorio (Entre 1-3 segundos). Los clientes tienen un tiempo de espera máximo que pueden tardar en ser atendidos antes de abandonar el local sin su café.

Cuando un camarero se libera de su trabajo escoge a el cliente que se encuentre primero en la lista (Organizada por orden de llegada), una vez el camarero se encuentra atendiendo a un cliente tarda un tiempo aleatorio en su preparación (Entre 1-20 segundos) si al finalizar el café el tiempo que ha llevado su preparación no es superior al máximo tiempo que está dispuesto a esperar un cliente este se marchará satisfecho, en caso contrario se marchará enfadado y sin su café.

2 Requisitos funcionales de la aplicación

Los requisitos funcionales del sistema son los siguientes:

- RF-1: Cada cliente se comportará como un hilo (Thread) independiente que simula su llegada a la cafetería.
- RF-2: Cada camarero se comportará como un hilo (Thread) que atiende a un cliente.
- RF-3: Cada cliente debe poder esperar su café durante el tiempo definido en tiempoEspera. Si el café no está listo en ese tiempo, el cliente se marcha sin recibirlo.
- RF-4: Los camareros deben tener un método prepararCafe(Cliente c) que simule la preparación del café mediante Thread.sleep() durante un tiempo aleatorio.
- RF-5: Si un cliente se marcha antes de que su café esté listo, el camarero debe continuar con el siguiente cliente disponible.
- RF-6: La simulación debe continuar hasta que todos los clientes hayan recibido su café o se hayan marchado por exceder su tiempo de espera.

3 Requisitos no funcionales

Los requisitos no funcionales del sistema son los siguientes:

- RNF-1: La aplicación debe poder manejar múltiples clientes y camareros de manera concurrente.
- RNF-2: La aplicación debe actualizar los estados de los clientes en la interfaz.
- RNF-3: La aplicación debe funcionar correctamente aun que varios clientes llegan al mismo tiempo.
- RNF-4: La interfaz debe ser clara y fácil de entender, mostrando visualmente el estado de cada cliente en tiempo real.

4 Casos de uso

A continuación se expone un diagrama general de casos de uso de la aplicación:

Caso de uso	Descripción
Inicio de la simulación	El usuario pulsa el botón de inicio para lanzar la simulación de la cafetería; se crean los hilos de clientes y camareros y se inicializan las listas de la UI.
Llegada de clientes	Cada cliente llega como un hilo independiente, se registra en la lista de espera y empieza a contar su tiempo máximo de espera.
Atención por camarero	Los camareros atienden a los clientes en orden de llegada, preparan el café (Simulado con Thread.sleep) y actualizan el estado del cliente en la interfaz.
Cliente atendido	Cuando el café está listo dentro del tiempo de espera del cliente, el cliente se traslada a la lista de completados en color verde.
Cliente insatisfecho	Si el tiempo de espera del cliente se supera antes de recibir su café, se traslada a la lista de completados en color rojo.
Finalización de la simulación	La aplicación termina la simulación automáticamente cuando todos los clientes han sido atendidos o se han marchado.

5 Historias de usuario

ID	Como...	Quiero...	Para...
HU-01	Usuario	Iniciar la simulación de la cafetería	Ver cómo los clientes y camareros interactúan en tiempo real
HU-02	Cliente	Llegar a la cafetería con un tiempo de espera definido	Intentar conseguir un café antes de que se agote mi paciencia
HU-03	Camarero	Atender a los clientes por orden de llegada	Tratar de preparar todos los cafés
HU-04	Cliente	Recibir mi café dentro del tiempo de espera	Estar satisfecho y abandonar la cafetería correctamente atendido
HU-05	Cliente	Marcharme si mi tiempo de espera se agota	Reflejar que no he sido atendido y liberar espacio en la lista de espera
HU-06	Usuario	Ver el estado de cada cliente en la interfaz	Comprender visualmente el funcionamiento de la simulación
HU-07	Sistema	Finalizar automáticamente la simulación	Indicar que todos los clientes han sido atendidos o se han marchado

6 Arquitectura de la Aplicación

La aplicación se organiza siguiendo una arquitectura por capas que separa la interfaz de usuario y la lógica (Similar al modelo vista-controlador).

- **Capa de interfaz (UI):** Vista diseñada con FXML.
- **Capa de lógica:** Clases, gestión de hilos...

7 Flujo de datos

1. La cafetería se abre (Se inician los hilos).
2. Los clientes llegan a la cafetería.
3. Los clientes son atendidos por los camareros.
4. Los clientes abandonan la cafetería atendidos correctamente o no.

8 Tecnologías utilizadas

8.1 Frontend:

- Lenguaje: FXML
- Tecnologías: JavaFX, Scene Builder

8.2 Backend:

- Lenguaje: Java
- Tecnologías: JavaFX

9 Clases e Interfaces

A continuación se describen las principales clases e interfaces que componen el programa.

9.1 Launcher.java

Inicia la aplicación y llama a HelloApplication.java.

9.2 HelloController.java

Actúa como el Main.java, en el se crean las instancias de camareros, clientes, la lista de clientes y camareros y se inician los hilos. Esta clase contiene las siguientes variables y funciones:

- `clientesEspera`: VBox que contendrá los clientes en espera de ser atendidos.
- `clientesAtendiendo`: VBox que contendrá los clientes que están siendo atendidos.
- `clientesAtendidos`: VBox que contendrá los clientes que ya han sido atendidos.
- `listaClientes`: ArrayList conteniendo todas las instancias de clientes.
- `listaCamareros`: ArrayList conteniendo todas las instancias de camareros.
- `inicioCafeteria`: Función que inicia la aplicación, se inicia en un nuevo hilo para no bloquear el hilo del UI.

- **actualizarEstadoCliente:** Recibe el nombre de un cliente y su estado de la clase Camarero.java o Cliente.java para actualizar las listas.
- **eliminarClienteLista:** Recibe un VBox(clientesEspera/clientesAtendiendo/clientesAtendidos) desde actualizarEstadoCliente para eliminarlo de las listas antes de cambiar el cliente de lista para evitar duplicados.

9.3 HelloApplication.java

Inicia la venta y llama a HelloController al ser llamado desde Launcher.java, además asigna el tamaño de la ventana.

9.4 Camarero.java

Clase que define a la entidad de Camarero, contiene un constructor con los atributos:

- **nombreCamarero:** String que determina el nombre del camarero (Su funcionalidad solo se aplica en el terminal).
- **listaClientes:** List<Cliente>, lista sincronizada entre todo el programa conteniendo los clientes.
- **HelloController:** Instancia de HelloController.

Además, contiene los siguientes métodos:

- **run:** Método heredado de Thread, que recorre la lista sincronizada, mantiene en espera al camarero en caso de estar vacía y cuando recibe un cliente lo envía al método prepararCafe.
- **prepararCafe:** Recibe un cliente desde run, establece un tiempo de preparación, comprueba si este es menor al máximo tiempo de espera de un cliente para comprobar si este se marcha con su café o no. Además, envía los datos necesarios a HelloController.actualizarEstadoCliente.

9.5 Cliente.java

Clase que define a la entidad Cliente, contiene un constructor con los atributos:

- **nombreCliente:** String que determina el nombre del cliente (Su funcionalidad solo se aplica en el terminal).
- **tiempoEspera:** Int que determina el tiempo de espera máximo que está dispuesto a esperar un cliente para ser atendido.
- **atendido:** Boolean que determina si un cliente ha sido atendido o no (Variable de control para saber si un cliente ha sido atendido o no).
- **listaClientes:** List<Cliente>, lista sincronizada entre todo el programa conteniendo los clientes.
- **HelloController:** Instancia de HelloController.

Además, contiene el siguiente método:

- **run**: Método heredado de Thread, informa de la llegada de un cliente a la cafetería y manda la información a `HelloController.actualizarEstadoCliente` y además elimina al cliente atendido de la lista.

9.6 hello-view.fxml (Interfaz de usuario)

Contiene el fondo del programa, las listas entre las que iteran los clientes y el botón de inicio de la aplicación, todo ello empleando FXML.

10 Bibliografía

References

- [1] Geeks for geeks. *Java notify() Method in Threads Synchronization with Examples*. Disponible en: <https://www.geeksforgeeks.org/java/java-notify-method-in-threads-synchronization-with-examples/>.
- [2] OpenAI. ChatGPT *Explicación de Platform.runLater* <https://chatgpt.com>